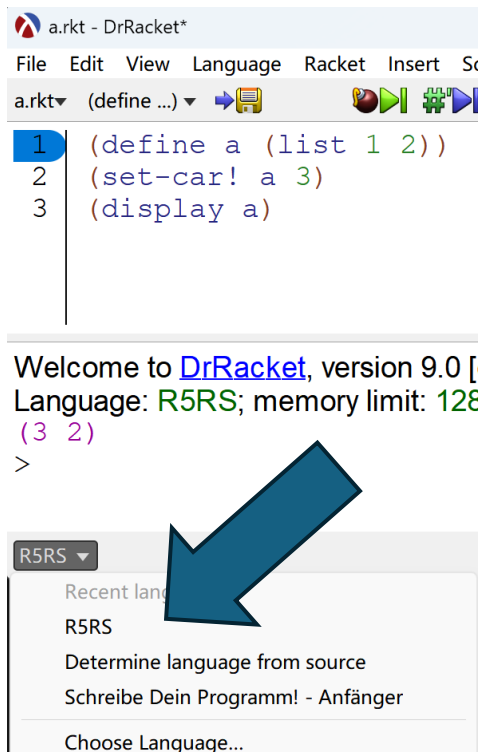


Computer Science 231
Laboratory 9 - More Objects and Mutating Lists in Scheme
Due: 10am Thursday April 2, 2026

During this lab you will continue to work with mutating lists. You will work on creating an object factory in Scheme called **make-dictionary** that you can use to create, maintain, and use a dictionary.

Be sure to name your file exactly as requested. Be sure to name your file exactly as requested. If you don't name your file exactly as requested, my auto tester won't run. If your code still contains any errors reported by DrRacket, you are very likely to receive a score of zero.

Note: set-car! and set-cdr! only work in **R5RS**. They will **not work in Racket**. Please select **R5RS (Revised⁵ Report on the Algorithmic Language Scheme)**



Background

All of the concepts that you need to complete these exercises have been covered during lecture. Use your notes and the reference material if you need a refresher. Or, ask me to review something with you. Remember that you are only using language tools that have been covered during class.

Refer to the *Background* section of Lab 7 if you want to review how objects are handled in Scheme.

As a reminder, here are some Scheme primitive forms relating to lists:

- **set!**. Sets the value of a variable to a new value.
- **set-car!**. Sets the **car** of an existing pair to a new value.
- **set-cdr!**. Sets the **cdr** of an existing pair to a new value.
- **case**. Use case to make your code more readable when testing many mutually exclusive conditions. We used the case statement in class to receive and distinguish messages given to the object. Remember that each case clause of a **case** special form is an implicit **begin**.
- **assoc**. Returns the pair associated with with the *key* in an association list. E.g. (**assoc** *key* alist)

Graded Exercises (17 marks)

In this exercise, you will create a Scheme procedure for creating **dictionary** objects.

Notes About Dictionaries:

- A dictionary is a data structure that stores data as a collection of key-value pairs where each key is unique.
- The key-value pairs in a dictionary are not necessarily sorted.
- A typically provides operations to *insert*, *update*, and *delete* key-value pairs, as well as an operation to *lookup* the value associated with a key. Other operations that are often provided with dictionaries is *size* (to return the total number of pairs) and the predicate *is-empty?* with the obvious interpretation.

In this exercise, you will create a procedure called **make-dictionary** that is a factory for creating **dictionary** objects. **make-dictionary** will return a lambda that will let you use it in

a manner similar to the way you use objects in other languages. Your **make-dictionary** procedure should not create any bindings at the global environment except for the binding of **make-dictionary** itself.

Your dictionary objects should be able to handle the following messages:

- **'insert** - This message indicates that a *key* and a *value* will be taken as arguments and a new key-value pair will be added to the dictionary.
- **'lookup** - This message indicates that a *key* will be taken as the argument. lookup will return the *value* of the key-value pair that matches the *key* provided as an argument.
- **'update** - This message indicates that a *key* and a *value* will be taken as arguments and the value of the key-value that matches the *key* will be updated to the *value* provided as an argument..
- **'size** - This message indicates an integer will be returned displaying the total number of key-value pairs contained in the dictionary.
- **'show** - This message indicates that the contents of the dictionary should be displayed in a formatted manner. The format should show one key-value pair per line with an arrow (->) pointing from the key to the value for each.
- Any other message will generate the error message **"Oops. Bad message."**

The following list of forms demonstrates how fractions are to be created and used and how I may test your code:

```
> (define fr-en (make-dictionary))
> (fr-en 'show)
Empty dictionary.
> (fr-en 'insert 'dog 'chien)
> (fr-en 'insert 'apple 'pomme)
> (fr-en 'insert 'hello 'bonjour)
> (fr-en 'show)
hello -> bonjour
apple -> pomme
dog -> chien
> (fr-en 'insert 'hi 'salut)
```

```
> (fr-en 'insert 'bye 'salut)
> (fr-en 'show)
bye -> salut
hi -> salut
hello -> bonjour
apple -> pomme
dog -> chien
> (fr-en 'lookup 'dog)
chien
> (fr-en 'lookup 'bye)
salut
> (fr-en 'lookup 'croissant)
Key not found.
> (fr-en 'lookup 'hello)
bonjour
> (fr-en 'update 'bye 'aurevoir)
> (fr-en 'insert 'soccer 'football)
> (fr-en 'show)
soccer -> football
bye -> aurevoir
hi -> salut
hello -> bonjour
apple -> pomme
dog -> chien
> (fr-en 'size)
6
>
```

Submitting your work:

Save your procedures in a file named LabX_GivenName_NNNNN.rkt

Where:

X is the lab number

GivenName is your preferred name

NNNNN is the last five digits of your student ID number

Submit the file directly through Moodle. You may submit it only one time before the deadline.

